The most general iterator in Ruby is each, which takes a block and invokes the block once for each element in the collection. In this case, we're using reverse_each, a shortcut method which invokes the block once for each element of the list, but in reverse order.

Enumerator methods can have different behavior beyond just executing the block of code. A Ruby programmer might use a different enumerator method called map to write the code more compactly. For example:

```
puts top_five.reverse.map { |word, count| "#{word}:  #{count}" }
```

The map applies its block to each element of the array in turn, returning a new array made up of the result of each invocation of the block.

Now the whole example looks like this:

tut_containers/word_freq/best_word_count.rb

```
require_relative "words_from_string"

raw_text = "The problem breaks down into two parts. First, given some text
as a string, return a list of words. That sounds like an array. Then, build
a count for each distinct word. That sounds like a use for a hash---we can
index it with the word and use the corresponding entry to keep a count."

word_list = words_from_string(raw_text)
counts = word_list.tally
sorted = counts.sort_by { |word, count| count }
top_five = sorted.last(5)
puts top_five.reverse.map { |word, count| "#{word}:  #{count}" }
```

produces:

```
a:  6
the:  3
count:  2
word:  2
sounds:  2
```

The map method is now taking each element of our top five array and converting it to a new array made of the strings that come as the result of executing the block.

Because each local variable is only used as the receiver of the next message, you could chain all the values together and get something like this:

tut_containers/word_freq/bester_word_count.rb

```
require_relative "words_from_string"

raw_text = "The problem breaks down into two parts. First, given some text
as a string, return a list of words. That sounds like an array. Then, build
a count for each distinct word. That sounds like a use for a hash---we can
index it with the word and use the corresponding entry to keep a count."

puts words_from_string(raw_text)
  .tally
  .sort_by { |word, count| count }
  .last(5)
  .reverse
  .map { |word, count| "#{word}:  #{count}" }
```

produces:

```
a:  6
```